

# Unit 4

# Advanced Git

By:

Name :Divya Thakur

UID : 28300

# Unit 4 Topics

- Source Code Management with git
- Comparison Commands
- Branching and Merging
- Rebasing
- Stashing
- Tagging

## Source Code Management with git

- Source Code Management (SCM) is a set of processes and tools used to manage and track changes in the source code of software projects.
- It involves version control, which is the practice of tracking and managing changes to the source code, as well as other related practices like branching and merging.\
- Common SCM tools include Git, Subversion, and Mercurial. These tools provide functionalities for version control, branching, merging, and other SCM practices.

- **SCM is crucial for software development projects for several reasons:**
- Collaboration
- History
- Reverting: If a change introduces a bug or causes other problems, SCM tools make it possible to revert the code back to a previous, stable state.
- Branching and Merging
- Conflict Resolution: When multiple developers work on the same part of the code, there is a risk of conflicts. SCM tools help in identifying and resolving these conflicts.

# Types of Source Code Management:

## **Centralized Version Control Systems (CVCS):**

- These systems use a central server to store all the files and histories of a project.
- Developers can check out the latest version or any previous version of the files from this central repository to their local machines.
- However, all the changes and new versions are recorded back in the central server.
- Examples of CVCS include:
  - Subversion (SVN)
  - Perforce
  - Microsoft Team Foundation Server (TFS)

- **Distributed Version Control Systems (DVCS):**

- These systems do not rely on a central server to store all the files and histories.
- Instead, every user has a complete copy of the entire repository, including the history, on their local machine.
- This allows developers to work offline and makes the repository more resilient because there are multiple copies of the repository.
- Examples of DVCS include:
  - Git
  - Mercurial
  - Bazaar

# Source code management best practices

- Commit often:
  - Each commit is a snapshot that the codebase can be reverted to if needed. Frequent commits give many opportunities to revert or undo work
- Ensure you're working from latest version :
  - SCM enables rapid updates from multiple developers.
- Make detailed notes
  - Each commit has a corresponding log entry.
- Review changes before committing
  - SCM's offer a 'staging area'.
- Use Branches
  - Branching is a powerful SCM mechanism that allows developers to create a separate line of development.
- Agree on a Workflow
  - By default SCMs offer very free form methods of contribution.

- Git is currently the most popular SCM tool, used by many open-source projects and companies worldwide.
- It is a distributed version control system, which means that every developer has a complete copy of the entire repository on their local machine.
- Git allows multiple developers to work on the same project simultaneously, and it provides functionalities for branching, merging, and conflict resolution.
- This enables a collaborative and efficient development process.



# Branching and Merging

- A branch in Git is a way to keep developing and coding a new feature or modification to the software and still not affecting the main part of the project.
- We can also say that branches create another line of development in the project.
- The primary or default branch in Git is the master branch .
- As soon as the repository creates, so does the main branch (or the default branch).
- Branches give you the freedom to independently work on different modules and merge the modules when you finish developing them.

# Different Operations On Branches

- Branches are the heart of Git.
- From the time of creating the repository until its deletion, everything will involve branches.
- **Create a Branch:**
  - This is the first step in the process, you can start on a default branch or create a new branch for the development.
- **Merge A Branch:**
  - An already running branch can merge with any other branch in your Git repository. Merging a branch can help when you are done with the branch and want the code to integrate into another branch code.
- **Delete A Branch:**
  - Deleting a branch can help when the branch has done its job, i.e., it's already merged, or you no longer need it in your repository for any reason.
- **Checkout A Branch:**
  - An already running branch can pull or checkout to make a clone of the branch so that the user can work on any of them.
  - Pulling a branch can help when you don't want to disturb the older branch and experiment on the new one.

- git branch:  
To view all the branches
- git branch -a :
  - To view branches including remote branches.
- git branch <branch\_name>:
  - Create a new branch
- git push -u origin prod:
  - command to update the local branch on the GitHub repository.
  - The -u flag adds the tracking reference for the prod branch so that your upcoming pushes do not require any extra arguments, and the local branch links to the remote branch.
- git checkout <BRANCH\_NAME >:
  - command to switch to the branch

- **git checkout -b <name\_of\_branch> :**
  - Create and Switch a branch with a single Git command:
  - The flag "b" here tells Git to create a new branch with the branch name succeeding in the flag.

# Deletion of branches

- Deletion of branches from our local systems and push them to the remote repository.
- `git branch -d <branch_name>`:
  - The "d" flag used here specifies that we intend to delete a branch.
  - Make sure you are not running the command on the same branch , checkout from the branch and then run the command
- `git branch -D <branch_name>`:
  - Command to force delete the branch without checking the merged status of the branch.
  - Doing so, Git won't give any warnings even if the branch does not merge in.

# Delete A Remote Branch

- Check the remote branches first by command.
  - `git branch -a`
- **`git push <remote_repo_name> --delete <branch_name>`:**
  - Command to delete a remote branch .
- To recover the deleted branches:
  - **`git reflog`**
  - Reflog is short for Reference Logs, which records the points when the updating of the branches and other references happened in the repository.
  - One can recover the branches by going back to a particular log.

## Practice Exercise:

- Create a branch locally by the name "new".
- Switch to the branch and make some changes by creating a new file etc.
- Commit these changes and look at the Git Logfile of the "new" branch.
- Push this branch to the remote repository.
- View all the remote and local branches to confirm.
- Delete this branch locally from Git.
- Delete this branch from a remote repository.

# Merge Branches

- A merge operation can be executed by typing the command:
- `git merge <branch_name>`
- The changes done by the `git merge` command can be undone through **git reflog** and **git hard reset** commands.



# How to merge branch in Git to another branch?

- Task :
- Merge a branch called "new" into another branch called the "master" branch.
- Before merging ,switch to the branch and log some changes into the branch "new".
- Commit these changes.
- Check the commit using oneline flag .
- Now switch to the branch to which we will merge the changes.
- Execute the following command to merge the branch "new" to the master branch.
- Recheck the log to see the merge.
- Once done, we can delete the branch.

# Comparison Commands

- The git diff command shows the code changes between two commits or between the current repository and an earlier commit.
- This command displays changes indicated by file headers and metadata for changed files
- The syntax for the git diff command is as follows::

`git diff`

- Git Diff Between two branches (Two Dots Method)

Syntax: `git diff branch1..branch2`

- This will show you all of the commits that branch2 has that branch1 doesn't have.

# Git tagging

- Tags in Git are the reference points in the Git history that denotes special events.
- You can tag any commit for any purpose you want.
- Tags can be associated with either the commit to which the HEAD points or a specific commit that the user likes.
- Tags are ref's that point to specific points in Git history.
- Tagging is generally used to capture a point in history that is used for a marked version release (i.e. v1.0.1).
- A tag is like a branch that doesn't change. Unlike branches, tags, after being created, have no further history of commits.

- Command to create a tag:

**git tag <tagname>**

- View/List Tags In Git:

**git tag**

- Push Tags to GitHub:

- Pushing the tags is similar to pushing anything on the remote repository.

**git push origin v1.4**

- Delete a tag:

**git tag -d v1**

- Checkout a tag :view the state of a repo at a tag. This puts the repo in a detached HEAD state.

**git checkout tagname**

# Git supports two different types of tags,

1. Annotated
2. lightweight tags

- Lightweight tags and Annotated tags differ in the amount of accompanying meta data they store.
- A best practice is to consider Annotated tags as public, and Lightweight tags as private.
- Annotated tags store extra meta data such as: the tagger name, email, and date.
- This is important data for a public release.
- Lightweight tags are essentially 'bookmarks' to a commit, they are just a name and a pointer to a commit, useful for creating quick links to relevant commits.

# Annotated tags

- Annotated tags are stored as full objects in the Git database.
- To reiterate, They store extra meta data such as: the tagger name, email, and date.
- Similar to commits and commit messages Annotated tags have a tagging message.
- Additionally, for security, annotated tags can be signed and verified with GNU Privacy Guard (GPG).
- Suggested best practices for git tagging is to prefer annotated tags over lightweight so you can have all the associated meta-data.

# Command for annotated tags

- Executing this command will create a new annotated tag identified with v1.5 :

```
git tag -a v1.5
```

- Tag with a message In Git:

```
git tag -a <tag_name> -m "<Message_for_commit>"<commit_hash_code>
```



# Lightweight tags

- Lightweight tags are created with the absence of the -a, -s, or -m options.
- Lightweight tags create a new tag checksum and store it in the .git/ directory of the project's repo.

**git tag v1.4-lw**

# Tagging old/Specific commits

- By default, git tag will create a tag on the commit that HEAD is referencing.
- Alternatively git tag can be passed as a ref to a specific commit.
- This will tag the passed commit instead of defaulting to HEAD.
- To gather a list of older commits execute the git log command.

```
git tag -a <tagname> <commit_hash>
```

- Note ,use git log to see the previous commits , then select the commit to which you wish to add a tag and then run the above command.

# Retagging/Replacing old tags

- If you try to create a tag with the same identifier as an existing tag, Git will throw an error like:
- fatal: tag 'v0.4' already exists
- Additionally if you try to tag an older commit with an existing tag identifier Git will throw the same error.
- In the event that you must update an existing tag, the -f FORCE option must be used.

**git tag -a -f <tagname> <commit\_hash>**

# Rebasing



Git Merge



Git Rebase

- ★ Imagine, you have a Master branch and a test branch(local branch)
- ★ The developer has finished his/her work in the test branch
- ★ But the master moved forward, while the code was being developed
- ★ Code being developed is related to the new commit added in master





Git Merge



Git Rebase



Therefore you want all the changes from master in feature.



Since, it is a local branch, you would want a cleaner or linear history, you decide to use git rebase

Syntax: **git rebase <source branch>**



## Rebasing

- Rebase is one of two Git utilities that specializes in integrating changes from one branch onto another.
- The other change integration utility is git merge.
- Rebasing is the process of moving or combining a sequence of commits to a new base commit.
- Rebasing is most useful and easily visualized in the context of a feature branching workflow.
- The primary reason for rebasing is to maintain a linear project history.
- Rebase itself has 2 main modes:
  - "manual/standard" and "interactive" mode.

- You have two options for integrating your feature into the main branch: merging directly or rebasing and then merging.
- The former option results in a 3-way merge and a merge commit, while the latter results in a fast-forward merge and a perfectly linear history.

# Standard

`git rebase <base>`

- This automatically rebases the current branch onto `[?]` base `[?]` , which can be any kind of commit reference (for example an ID, a branch name, a tag, or a relative reference to HEAD).



# Interactive Rebasing

- Running git rebase with the -i flag begins an interactive rebasing session.
- Instead of blindly moving all of the commits to the new base, interactive rebasing gives you the opportunity to alter individual commits in the process.
- This lets you clean up history by removing, splitting, and altering an existing series of commits.

**git rebase --interactive <base>**

- This rebases the current branch onto [?] base [?] but uses an interactive rebasing session.
- This opens an editor where you can enter commands for each commit to be rebased.
- These commands determine how individual commits will be transferred to the new base.

- Interactive rebasing gives you complete control over what your project history looks like.
- Most developers like to use an interactive rebase to polish a feature branch before merging it into the main code base.
- The real power of interactive rebasing can be seen in the history of the resulting main branch.

# Task

- Create a new branch based off of main  
git checkout -b new master
- Add file and edit it.
- commit the changes with message :  
git commit -a -m "Adds new feature"
- Rebase in both the modes and see the difference .

# Git Stashing

- The git stash command takes your uncommitted changes (both staged and unstaged), saves them away for later use, and then reverts them from your working copy.

`git stash`

**eg : `git stash push -m "WIP: notes edit"`**

- List all the stashes:

`git stash list`

- To see whats inside stash :

`git stash show -p stash@{0}`

- To revert all the changes/reapply previously stashed changes with git stash pop.(Reapply or Restore )

### **git stash pop**

Popping your stash removes the changes from your stash and reapplies them to your working copy.

Alternatively, you can reapply the changes to your working copy and keep them in your stash with

### **git stash apply**

**If you used git stash apply instead of pop, you can remove stash manually**

- Drop or Clear Stashes :

**git stash drop stash@{0}**

- To remove all :

**git stash clear**

- Adding the -u option (or --include-untracked) tells git stash to also stash your untracked files

Git stash - u

- To provide a bit more context, it's good practice to annotate your stashes with a description, using :

git stash save "message"

- You can view a summary of a stash with  
git stash show

- Delete a particular stash:

`git stash drop stash@{1}`

- Delete all the stashes:

`git stash clear`

- Creating a branch from your stash:

you can use `git stash branch` to create a new branch to apply your stashed changes to:

`git stash branch branchname stash@{1}`

# Task on Stashing:

- Scenario: You're Editing a Text File and Need to Switch Branches
- Let's say you're working in a folder called demo-stash.
- **Create and Commit a File.**
- **Now you edit the file but don't commit.**
- Check status.
- Try to Switch Branches( create a new branch and checkout).
- Stash the Changes.
- Now check status.(Your changes are saved safely in the stash)
- Switch Branches and Do Other Work(create a new file and commit changes).
- Go Back and Restore Stash.